

Retriever、Vector Store 与混合检索

DOC_ID: DOC-RET-001

说明：本文档为 RAGLab 项目生成的教学用合成语料，用于分块、检索、重排、引用和评测实验。它不是 LangChain/LangGraph 官方文档，也不应被当作 API 版本说明。文中的稳定标识（DOC_ID、SECTION_ID）用于构建可重复的检索评测集。

1. Retriever 与 Vector Store 的区别

SECTION_ID: RET-BASIC-01

Retriever 的抽象可以概括为：输入查询字符串，返回一组 Document。它描述的是检索行为，而不是数据必须存放在哪里。Vector Store 负责存储向量并执行相似度搜索，只是 Retriever 的一种数据来源。

因此，下列组件都可以实现 Retriever 接口：

- 向量相似度检索；
- BM25 关键词检索；
- 数据库全文检索；
- 多个检索器的融合；
- 父子文档检索；
- 带元数据过滤的检索。

把 Retriever 与 Vector Store 混为一谈，会限制对混合检索和路由检索的理解。

2. Dense Retrieval

SECTION_ID: RET-DENSE-01

Dense Retrieval 将查询和文档块映射到向量空间，通过距离或相似度寻找语义接近的内容。它擅长处理用户表述与文档用词不同的情况。

例如文档中写“持久化图状态”，用户提问“中断后怎样恢复之前的执行进度”，语义检索可能仍能建立关联。

Dense Retrieval 的弱点包括：

- 对精确类名、编号和缩写不一定敏感；
- Embedding 模型变化会影响索引；
- 相似度高不等于能够回答问题；
- 大量相似重复块会挤占 Top-K。

3. Sparse Retrieval 与 BM25

SECTION_ID: RET-BM25-01

BM25 依赖词项匹配和词频统计，适合类名、错误码、接口字段、产品编号等精确查询。例如 `Command(resume=...)`、`thread_id` 或 `ERR-1042` 这类查询通常能从关键词检索中获益。

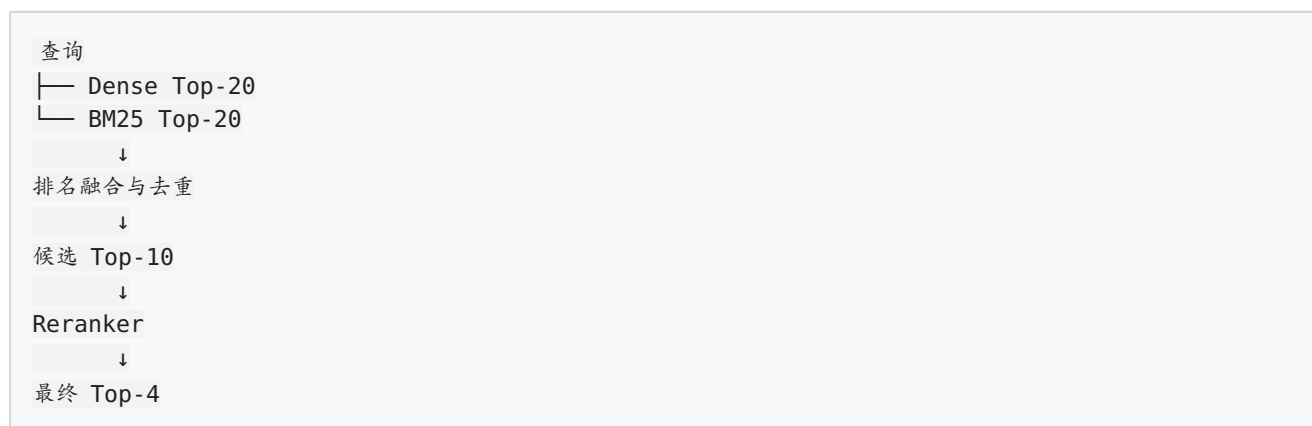
BM25 的不足是难以处理同义表达。用户问“状态恢复机制”，文档只写“checkpoint persistence”时，关键词重叠可能很少。

4. Hybrid Retrieval

SECTION_ID: RET-HYBRID-01

混合检索将 Dense 和 Sparse 的结果进行融合。常见做法不是直接比较两种分数，因为两者的分数尺度不同，而是使用排名融合，例如 Reciprocal Rank Fusion。

一个简化流程为：



混合检索适合技术文档，因为技术问题同时包含语义表达和精确术语。

5. Metadata Filtering

SECTION_ID: RET-META-01

元数据过滤发生在检索阶段，用于缩小候选范围。例如：

```
{
  "corpus": "langgraph",
  "version": "1.x",
  "language": "zh-CN",
  "source_type": "tutorial"
}
```

如果用户明确询问 LangGraph，就可以先限制 `corpus=langgraph`，再做向量检索。过滤过严会漏召回，过滤过宽则会引入噪声，因此需要在测试集上比较。

6. Top-K 与多样性

SECTION_ID: RET-TOPK-01

增大 Top-K 通常会提高召回机会，但也会增加噪声、上下文长度和 Token 成本。MMR 通过平衡相关性和多样性，减少多个高度相似块同时进入上下文。

评测时应分别记录：

- Hit@K：至少一个相关文档是否进入前 K；
- Recall@K：所有相关文档中找回了多少；
- MRR：第一个相关结果排在多靠前；
- Context Precision：进入上下文的块有多少真正相关。